

CPE334 Lab 1 - TokTickIT Full-Stack Vertical Slice

Individual submission prepared from the student's own repository history and runtime evidence.

Student	Rattathamnoon Buranapattana
Student ID	67070501037
GitHub	TauForge
Repository	https://github.com/TauForge/TokTickIT
Reviewer / partner	Sittichai Phirompha (Bank) - 67070501074 - Bank848
AI / LLM	OpenAI Codex with GPT-5

Answer Part 1: Git Use and Engineering Workflow

This part records the repository, Project board, issue sequence, PR sequence, final main history, repository structure, README, .gitignore, and reciprocal review evidence.

Repository, Project, Issue, and PR URLs

Item	Actual link / record
Repository	https://github.com/TauForge/TokTickIT
GitHub Project	TokTickIT Individual Sprints - https://github.com/users/TauForge/projects/7
Issues	#1 Foundation, #2 API health check, #3 Category seed, #4 Category list
Feature PRs	#5 feature/1-project-foundation; #6 feature/2-health-check; #7 feature/3-category-seed; #8 feature/4-category-list
Promotion PR	#9 lab1-staging -> main
Documentation PR	#10 feature/lab-01-documentation -> main (open; documentation only)

Issue titles and final Project status

Issue	Title	Status
#1	Set up the TokTickIT project foundation	Done
#2	Implement the API health check	Done
#3	Create and seed IT request categories	Done
#4	Display the IT request category list	Done

The Project Board field options are Backlog, Specified, Started, PR Review, Fixing, and Done, in that order. The live board snapshot shows all four Issues in Done and zero items in the earlier workflow columns.

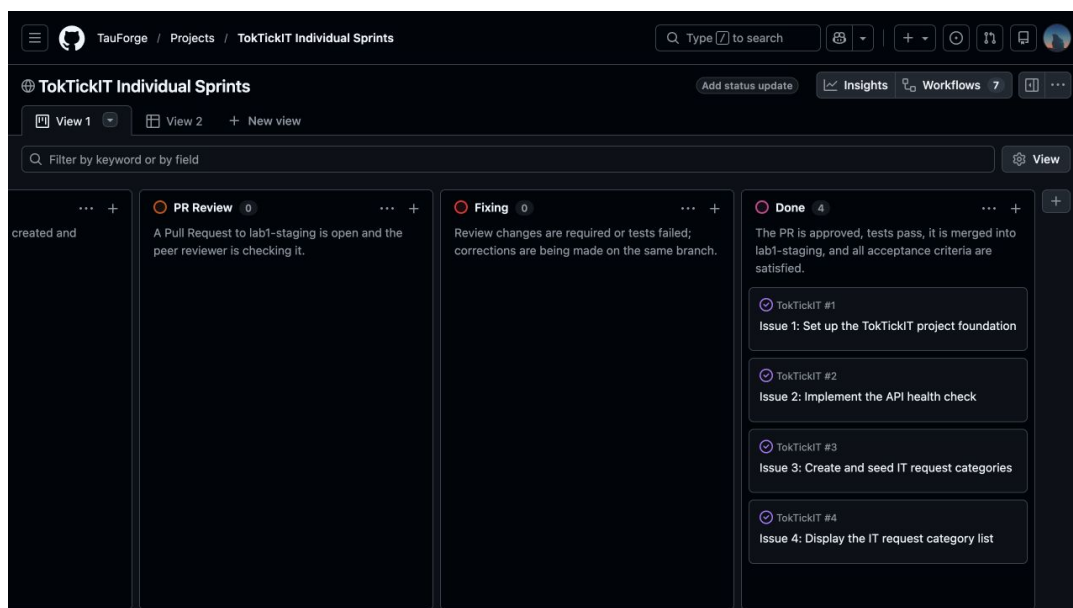


Figure 1. GitHub Project board - Done contains Issues #1, #2, #3, and #4.

Answer Part 1 continued: Main History and Repository Evidence

Main branch commit history

The final integration commit is 6f3c281 on origin/main. The history below shows feature work merged into lab1-staging and then promoted to main: PR #5 -> #6 -> #7 -> #8 -> #9.

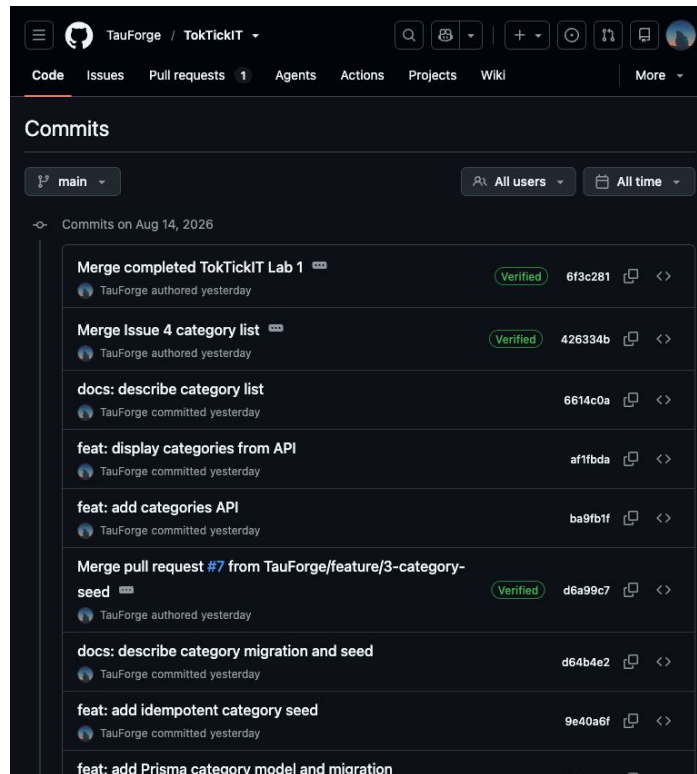


Figure 2a. Main history from the final promotion down through the category and health work.

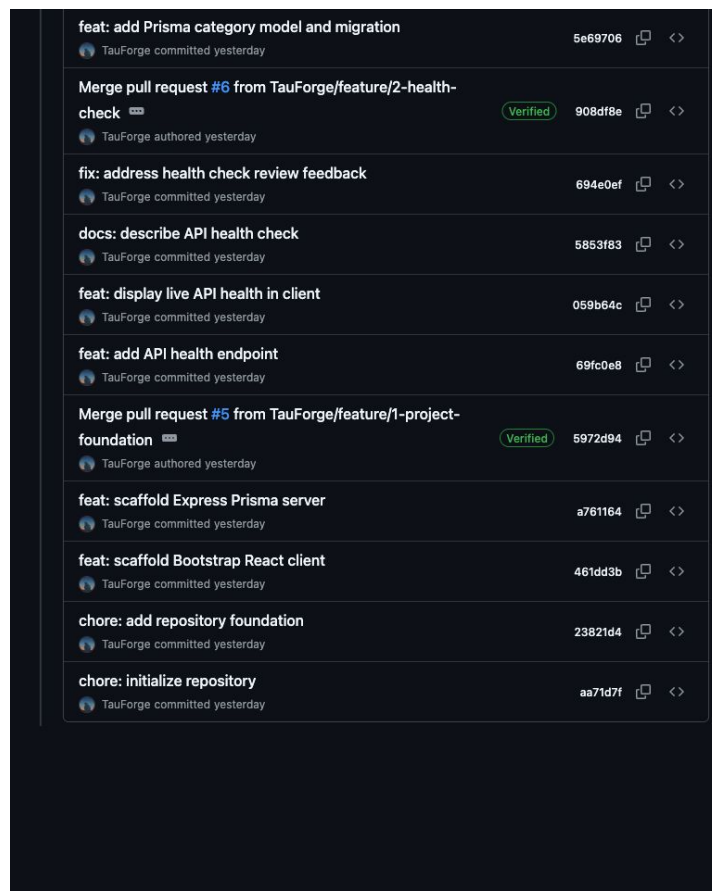


Figure 2b. Main history continues to the foundation commits and initial commit aa71d7f.

Directory structure on origin/main

```
toktickit/
|-- client/
|   |-- src/ (App.tsx, main.tsx, vite-env.d.ts)
|   |-- tests/lab-01/App.test.tsx
|   |-- .env.example
|   |-- package.json
|-- server/
|   |-- prisma/ (schema.prisma, seed.ts, migrations/)
|   |-- src/ (app.ts, index.ts, prisma.ts)
|   |-- tests/lab-01/categories.test.ts
|   |-- tests/lab-01/foundation.test.ts
|   |-- package.json
|-- docs/lab-01/
|   |-- ai_use.md
|   |-- reviewer.md
|   |-- tests.md
|-- README.md
|-- .gitignore
|-- .env.example
|-- docker-compose.yml
|-- package.json
```

The package-level test folders are `client/tests/lab-01/` and `server/tests/lab-01/`, alongside the required `docs/lab-01/` evidence files.

README.md (rendered)

TokTickIT

TokTickIT is an IT service desk application being built through the CPE334 individual sprint workflow.

Current branch scope: Issue 1 foundation, Issue 2 API health check, Issue 3 category seed, and Issue 4 category list.

Stack: React + TypeScript + Vite + Bootstrap; Node.js + Express + TypeScript; PostgreSQL + Prisma; Vitest + Supertest.

Setup:

```
cp .env.example server/.env
cp client/.env.example client/.env
npm run install:all
docker compose up -d db
npm --prefix server run prisma:generate
npm --prefix server run prisma:validate
npm --prefix server run prisma:migrate:deploy
npm --prefix server run prisma:seed
```

Run:

```
npm run dev:server
npm run dev:client
```

Test and build:

```
npm test
npm run build:client
npm run build:server
```

The client shows the TokTickIT heading, Bootstrap foundation card, backend API status, and four seeded IT request categories. No later application features are implemented.

.gitignore (rendered)

```
node_modules/
.env
.env.*
!.env.example
dist/
build/
coverage/
.DS_Store
*.log
```

Answer Part 1 continued: Peer Review Evidence

Reviews received on TauForge/TokTickIT

PR	Branch / target	Evidence and result
#5	feature/1-project-foundation -> lab1-staging	Bank848 approved the clean Issue 1 foundation. Merged 5972d94.
#6	feature/2-health-check -> lab1-staging	Bank848 requested configurable API URL and timeout. TauForge replied, fixed both in 694e0ef, and resolved the thread. Merged 908df8e.
#7	feature/3-category-seed -> lab1-staging	Bank848 approved the Prisma model, migration, and idempotent seed. TauForge replied confirming the criteria. Merged d6a99c7.
#8	feature/4-category-list -> lab1-staging	Bank848 approved the API, UI states, validation, and tests. Merged 426334b.
#9	lab1-staging -> main	Bank848 approved the final promotion of reviewed Issue 1-4 work. Merged 6f3c281.

PR #6 review screenshot

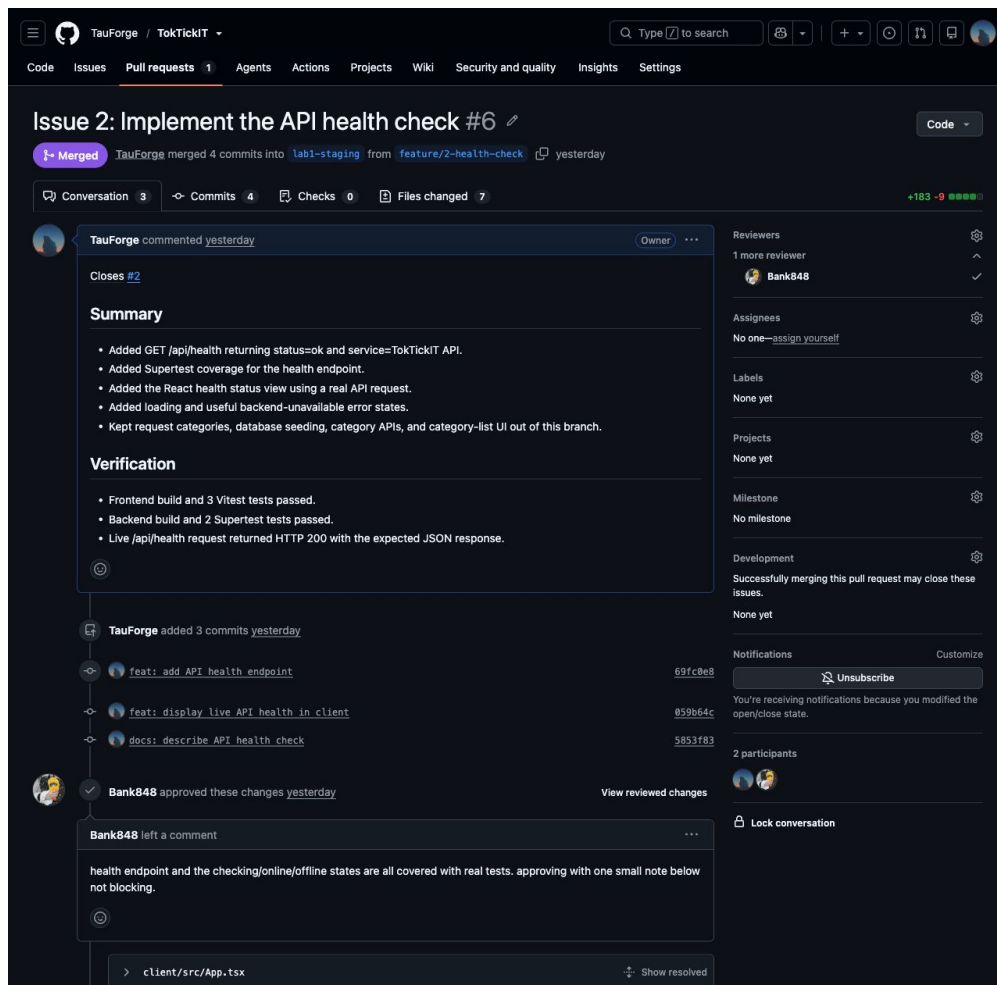


Figure 3. Bank848 approved PR #6 and the page records the follow-up review note and merged fix commit 694e0ef.

Reviews given to Sittichai Phirompha (Bank848)

Partner PR	TauForge review comment	Result
#7	LGTM. Checked the schema, migration, seed, and README. Everything looks good and matches the changes in this PR. No issues from my side. Approved.	Merged into lab1-staging
#8	LGTM. Checked the API, UI, and tests. Everything looks good and matches the requirements of this PR. No issues from my side. Approved.	Merged into lab1-staging
#9	Checked the changes and the lab documentation. Everything looks good and the previous PR changes are included correctly. No issues from my side. Approved.	Merged into main

Partner repository: <https://github.com/Bank848/toktickit>

Partner PR #7 screenshot

The screenshot shows a GitHub Pull Request (PR) #7 titled "Create and seed IT request categories #7" in the repository "Bank848 / toktickit". The PR is in the "Merged" state, having been merged into the "lab1-staging" branch from the "feature/3-category-seed" branch. The PR description states: "Adds the Prisma Category model (id, unique name, createdAt), migration, and an idempotent seed script for Account and Access, Hardware, Software, and Network. No API route or UI yet — GET /api/categories and the category-list UI are Issue #4." The PR is reviewed by "TauForge" (Sittichai Phirompha), who has left a comment: "LGTM. Checked the schema, migration, seed, and README. Everything looks good and matches the changes in this PR. No issues from my side. Approved." The PR is also reviewed by "ArmyC". The PR is assigned to "Bank848". The right sidebar shows the "Reviewers" section with "ArmyC" and "TauForge", the "Assignees" section with "Bank848", and the "Labels" section with "None yet". The "Development" section shows a message: "Successfully merging this pull request may close these issues." The "Notifications" section shows a button to "Unsubscribe" and a message: "You're receiving notifications because you commented." The "Participants" section shows 2 participants: "Bank848" and "TauForge".

Figure 4. The partner repository records TauForge's written approval on Bank848 PR #7 before merge.

All review records are linked in <docs/lab-01/reviewer.md>. The document identifies both students, both GitHub usernames, student IDs, review comments, responses, and merge results.

Answer Part 2: Tests

Test files and coverage

Test file	Tool	Coverage
client/tests/lab-01/App.test.tsx	Vitest + Testing Library	Foundation heading and Bootstrap marker; health success, unavailable backend, timeout; category success and error states.
server/tests/lab-01/foundation.test.ts	Supertest	Foundation route and GET /api/health response.
server/tests/lab-01/categories.test.ts	Supertest + Vitest	GET /api/categories response, selected id/name fields, and predictable id ASC order.

Passing output from main verification run

```
> npm test
> tokTickIT@0.0.0 test
> npm run test:client && npm run test:server

client: tests/lab-01/App.test.tsx (6 tests) - passed
Test Files 1 passed (1)
Tests 6 passed (6)

server: tests/lab-01/categories.test.ts (1 test) - passed
server: tests/lab-01/foundation.test.ts (2 tests) - passed
Test Files 2 passed (2)
Tests 3 passed (3)

> npm run build:client
TypeScript project build and Vite production build - passed

> npm run build:server
Backend TypeScript build - passed
```

The Supertest suite was rerun with the local network permission needed by the test server. The initial sandbox-only run was blocked by an environment EPERM while opening a port; it did not produce assertion failures.

Acceptance-criteria mapping

- Issue 1: Express and React/Vite/Bootstrap foundation are covered by the server foundation test and client foundation assertions.
- Issue 2: health success, backend unavailable, configurable API URL, and five-second timeout are covered by server and client tests.
- Issue 3: Prisma Category schema, migration, and idempotent seed provide the four IT request categories.
- Issue 4: the Prisma-backed categories route returns id/name in ID order, while the UI validates the response and exposes loading/error states.

Live API output

```
GET http://localhost:3000/api/health
HTTP/1.1 200 OK
{"status":"ok","service":"TokTickIT API"}

GET http://localhost:3000/api/categories
HTTP/1.1 200 OK
[{"id":1,"name":"Account and Access"}, {"id":2,"name":"Hardware"},
 {"id":3,"name":"Software"}, {"id":4,"name":"Network"}]
```

Answer Part 3: AI Use and Reflection

Tool and model

I used OpenAI Codex with GPT-5 as an AI coding assistant. I remained responsible for repository decisions, GitHub actions, review communication, test verification, and the final submission content. No ChatGPT or Codex co-author trailer was added to commits.

Selected prompts from the project workflow

Prompt	Use and reflection
Read both PDFs in full before doing anything.	Established the required order, branch policy, issue workflow, and evidence before code.
Create the public repo, main, and lab1-staging.	Turned the labsheet workflow into a concrete repository setup and kept work on feature branches.
Continue to step 3	Started Issue 1 only after the planning board and four issues were ready, keeping the foundation scoped.
Create the PR for Issue 2	Used the accepted Issue 1 result as the base for the next issue and linked the PR to its issue.
Review PR 6, fix feedback, reply, and merge.	Converted peer feedback into a configurable API URL, timeout handling, and an additional test.
Create the PRs for Issues 3 and 4	Repeated the issue -> branch -> review -> merge sequence without mixing later scope into earlier branches.
Merge and work on the documents	Switched from application delivery to test, review, AI, README, and final evidence work.
make it mine version using my friend PDF as an example	Used the example for organization only, replacing identity, links, review evidence, test results, and screenshots with my own facts.

Reflection

The most useful prompts stated constraints and acceptance criteria explicitly. A short prompt such as "Continue to step 3" worked because the earlier conversation already established the exact branch and issue scope. When a reviewer raised a concern on PR #6, a precise description of the wrong behavior made it possible to implement and verify the fix rather than only changing the visible message. For the final document, I checked every required item against the labsheet and used actual GitHub and local runtime evidence instead of copying the partner's submission.

Answer Part 4: App Demo

The frontend was opened at <http://localhost:5173> while the backend ran at <http://localhost:3000> against the seeded PostgreSQL database. The two screenshots below are browser captures from the same local application.

Success case: health and categories loaded

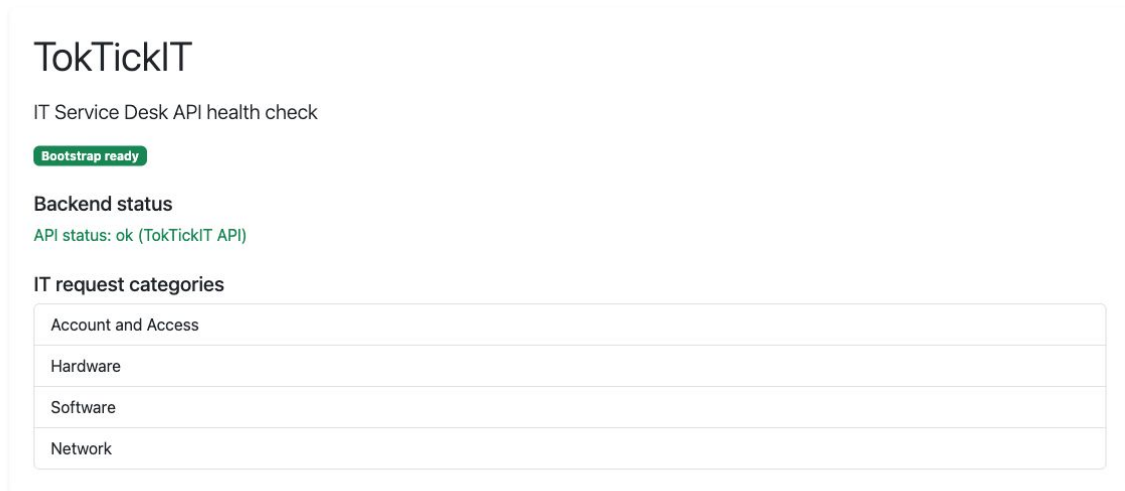


Figure 5. TokTickIT displays API status ok and all four categories: Account and Access, Hardware, Software, and Network.

Failure case: backend stopped

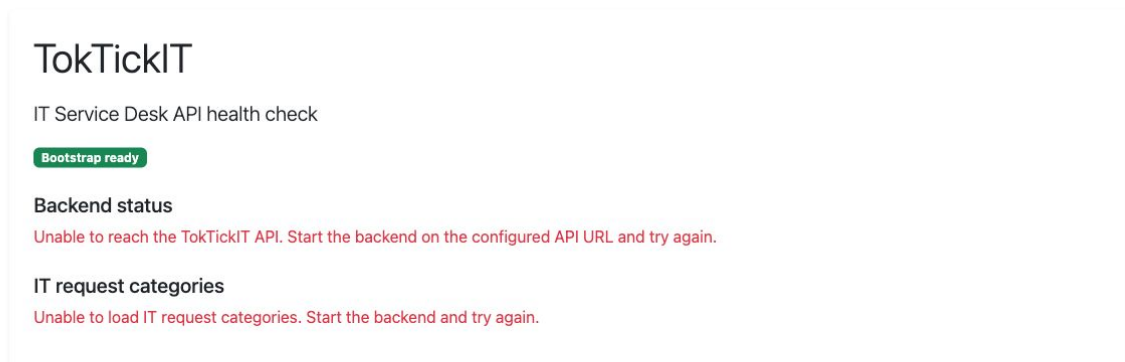


Figure 6. With the backend stopped, the app shows the required error/offline state for both health and categories.

The backend was restarted after the failure capture and the success state was verified again. The final PDF is the single submission artifact; the Markdown files in docs/lab-01/ are the rendered source records kept with the project.